

Configurazione dei sistemi esterni

1. Supabase

Database

Opzione CLI:

```
npx supabase login  
npx supabase link --project-ref IL_TUO_PROJECT_REF  
npx supabase db push
```

In alternativa, copiare il contenuto di `supabase/migrations/202609160001_findme.sql` nel **SQL Editor** ed eseguirlo.

Le migrazioni creano tabelle, anagrafica ricevitori, relazioni di pairing, replica Realtime e policy RLS. La migrazione `202609170002_tracking_performance.sql` aggiunge impostazioni adattive, lease temporanei e la RPC protetta per il percorso storico; la successiva `202609170003_tracking_policy_fix.sql` rende la lettura RLS non ricorsiva. Non disabilitare RLS. La migrazione `202609180001_delete_location_history.sql` abilita la cancellazione dello storico soltanto per trasmettitori associati a un ricevitore posseduto dall'utente autenticato. La migrazione `202609200001_command_recovery.sql` aggiunge l'intervallo globale del polling comandi usato come fallback quando Supabase Realtime non risponde. Le migrazioni `202609200002_voice_messages_schema.sql` e `202609200003_voice_messages_rpc.sql`, con la correzione isolata `202609200004_voice_storage_policy_fix.sql`, creano il bucket privato voice-messages, le policy RLS e il comando atomico per consegnare messaggi vocali. La migrazione `202609240001_receiver_service_configs.sql` abilita override LiveKit per ricevitore senza memorizzare credenziali nel database. Non rendere pubblico il bucket. Le migrazioni `202609290002_online_interval_2s.sql`, `202609290003_text_messages_schema.sql` e `202609290004_text_messages_rpc.sql` aggiungono l'intervallo online di due secondi e la coda persistente dei messaggi testuali con RPC atomica. Le migrazioni `202610010001_geofence_radius_options.sql` e `202610010002_receiver_access_configuration.sql` aggiungono i raggi area da 10/25 metri e la modifica owner-only di domanda e risposta.

Identità automatica

In **Authentication > Providers > Anonymous Sign-Ins** abilitare gli accessi anonimi. Le app creano e conservano automaticamente una sessione tecnica: l'utente non deve inserire email o password.

Il ricevitore mostra un codice univoco di 10 caratteri. Inserendolo nel trasmettitore si crea la relazione uno-a-molti in `receiver_transmitters`.

Edge Function LiveKit

Creare prima il progetto LiveKit, poi:

```
npx supabase secrets set `
  LIVEKIT_URL=wss://IL_TUO_PROGETTO.livekit.cloud `
  LIVEKIT_API_KEY=... `
  LIVEKIT_API_SECRET=...
npx supabase functions deploy livekit-token
npx supabase functions deploy pair-device
npx supabase functions deploy receiver-access
```

SUPABASE_URL e SUPABASE_ANON_KEY sono disponibili automaticamente nelle Edge Functions ospitate. Le funzioni verificano il JWT anonimo, la proprietà del dispositivo e la relazione di pairing prima di associare dispositivi o emettere un token LiveKit.

LiveKit dedicato per un ricevitore

Senza una riga in `receiver_service_configs`, il ricevitore usa il progetto LiveKit condiviso configurato sopra. Per assegnargli un progetto dedicato, creare due Edge Secrets con nomi univoci e non inserirne mai i valori in SQL:

```
npx supabase secrets set `
  CLIENTE_ACME_LIVEKIT_API_KEY=... `
  CLIENTE_ACME_LIVEKIT_API_SECRET=...
```

Dal SQL Editor, usando un ruolo amministrativo, salvare URL e soli nomi dei secret:

```

insert into public.receiver_service_configs (
  receiver_id,
  livekit_url,
  livekit_api_key_secret_name,
  livekit_api_secret_secret_name
) values (
  'UUID_DEL_RICEVITORE',
  'wss://CLIENTE_ACME.livekit.cloud',
  'CLIENTE_ACME_LIVEKIT_API_KEY',
  'CLIENTE_ACME_LIVEKIT_API_SECRET'
)
on conflict (receiver_id) do update
set livekit_url = excluded.livekit_url,
    livekit_api_key_secret_name = excluded.livekit_api_key_secret_name,
    livekit_api_secret_secret_name = excluded.livekit_api_secret_secret_name,
    updated_at = now();

```

La tabella non è accessibile ai client anonimi o autenticati. La Edge Function risolve il ricevitore dalla relazione di pairing e legge dinamicamente i secret. Per ruotare le credenziali mantenendo gli stessi nomi, aggiornare soltanto gli Edge Secrets. Per tornare al provider condiviso:

```

delete from public.receiver_service_configs
where receiver_id = 'UUID_DEL_RICEVITORE';

```

Domanda di accesso

Ogni ricevitore può avere access_question e access_answer_hash. La risposta viene normalizzata e verificata tramite bcrypt lato database; non viene mai salvata in chiaro né restituita alle app. La funzione receiver-access espone soltanto la domanda e l'esito della verifica al trasmettitore associato.

Retention

La funzione SQL delete_expired_findme_data() mantiene 30 giorni di posizioni e 7 giorni di comandi. Con Supabase Cron/pg_cron pianificarla una volta al giorno:

```

select cron.schedule(
  'findme-retention',
  '15 3 * * *',
  $$select public.delete_expired_findme_data()$$
);

```

2. LiveKit Cloud

1. Creare un progetto su LiveKit Cloud.
2. Copiare WebSocket URL, API key e API secret.
3. Mettere soltanto l'URL in `local.properties`.
4. Salvare key e secret nei Supabase secrets come indicato sopra.

Non occorre creare stanze manualmente: vengono create alla prima connessione con nome `receiver-<receiver_uuid>-device-<device_uuid>`. Firebase resta condiviso: i token FCM sono già isolati per `receiver_id` e non usano la configurazione LiveKit.

3. Mappe gratuite

Il ricevente usa MapLibre Native con le mappe vettoriali OpenFreeMap basate su OpenStreetMap. Non servono account, carta di credito o API key.

Lo stile configurato è:

```
https://tiles.openfreemap.org/styles/liberty
```

L'istanza pubblica è gratuita e include l'attribuzione richiesta. Non offre uno SLA: per un futuro impiego critico sarà possibile ospitare OpenFreeMap autonomamente senza modificare il database.

4. Device Owner sul trasmettitore

La modalità Device Owner consente all'app dedicata di concedere i permessi e riavviare il foreground service dopo boot. Il telefono deve essere privo di account, normalmente appena ripristinato.

Per sviluppo:

```
adb install -r transmitter\build\outputs\apk\debug\transmitter-debug.apk
adb shell dpm set-device-owner `
    it.xcc.findme.transmitter/.FindMeDeviceAdminReceiver
```

Nel provisioning QR inserire il codice del ricevitore negli admin extras:

```
{
  "android.app.extra.PROVISIONING_ADMIN_EXTRAS_BUNDLE": {
    "findme_receiver_code": "CODICE_RICEVITORE"
  }
}
```

FindMeDeviceAdminReceiver salva il codice al completamento del provisioning e l'app esegue il pairing senza mostrare una schermata preliminare. La prima schermata visibile è quindi sempre la domanda di accesso.

Il comando ADB `dpm set-device-owner` non supporta admin extras: in sviluppo il dispositivo deve essere già associato nel database oppure va usato il provisioning completo. Build firmata, checksum, generazione locale del QR e procedura sul telefono sono descritti in [QR PROVISIONING.md](#).

Poi premere almeno una volta **Avvia monitoraggio**. Il flag viene conservato e BootReceiver ripristina il servizio ai riavvii.

Per rimuovere il Device Owner durante lo sviluppo potrebbe essere necessario un factory reset; verificare questa procedura sul dispositivo di test prima del provisioning.

5. Impostazioni del telefono

- attivare lo switch **Nessuna restrizione batteria** in FindMe Trasmettitore;
- sui produttori che lo prevedono, abilitare l'avvio automatico;
- su Xiaomi/MIUI/HyperOS aprire anche le informazioni app e scegliere **Risparmio batteria > Nessuna restrizione**: la whitelist Android standard non disabilita sempre il gestore energetico proprietario;
- mantenere rete dati/Wi-Fi disponibile;
- concedere camera, microfono e posizione;
- concedere posizione "Sempre" se si vuole tracciare anche fuori dalla sessione.

Android continuerà a mostrare notifica foreground e indicatori camera/microfono.

6. Watchdog

Il trasmettitore usa l'heartbeat configurato dal ricevitore (30/60/90/120 secondi). Il ricevitore considera offline il dispositivo dopo due intervalli. La resilienza è composta da:

- START_STICKY per la ricreazione del servizio;
- un solo client Supabase per processo, così attività, servizio e FCM non competono nella rotazione del refresh token;
- refresh esplicito della sessione e rinnovo dei canali ogni 15 minuti;
- timeout di 20 secondi per sessione, heartbeat e upload posizione;
- retry esponenziale da 5 a 60 secondi, anticipato quando Android segnala il ritorno della rete;
- watchdog che ricrea tutti i flussi se heartbeat, autenticazione o Realtime si bloccano;
- retry persistente anche sul ricevitore e messaggi tecnici rimossi dopo il recupero;
- connessione LiveKit solo quando audio, video o schermo sono richiesti;
- cache locale delle ultime impostazioni tracking valide;

- lease rapido di 90 secondi, rinnovato ogni 20 secondi e con fallback offline;
- BootReceiver quando l'app è Device Owner;
- stato online/offline derivato dal timestamp Supabase.

Una perdita reale di Internet rende inevitabilmente il dispositivo temporaneamente offline. Al ripristino della rete non è necessario riaprire l'app né disattivare e riattivare il monitoraggio.

7. Mirroring schermo

Il primo setup del trasmettitore mostra la conferma Android MediaProjection. La cattura resta pronta nel foreground service, ma non viene pubblicata e non consuma participant-minutes LiveKit finché **Schermo** è OFF sul ricevitore.

MediaProjection non sopravvive a reboot, aggiornamento APK, arresto del processo o revoca dalla notifica di sistema. In questi casi la notifica FindMe apre la dashboard, dove **Riattiva** ripresenta il consenso. Contenuti DRM, finestre con FLAG_SECURE e alcune schermate di sistema possono apparire nere. L'audio interno non è acquisito.

Per i messaggi testuali in primo piano, autorizzare inoltre **Mostra sopra altre app** dalla dashboard del trasmettitore. Il permesso Android SYSTEM_ALERT_WINDOW non è pre-concedibile dal Device Owner. Se manca, il messaggio resta pendente e viene mostrato automaticamente quando l'utente lo concede.

FCM non è necessario per controllare camera e microfono sui telefoni Device Owner e non può aggirare i vincoli Android sui telefoni standard. È invece usato per consegnare al ricevitore gli avvisi area descritti di seguito.

8. Firebase Cloud Messaging per gli avvisi area

Gli avvisi di uscita area usano FCM e arrivano al ricevitore anche quando l'app non è aperta. Firebase Cloud Messaging non richiede un piano a pagamento.

1. Creare un progetto nella Firebase Console.
2. Aggiungere un'app Android con package `it.xcc.findme.receiver`.
3. Scaricare `google-services.json` e copiarlo in `receiver/google-services.json`. Il file è escluso da Git; senza di esso l'app compila, ma registra nei log che le notifiche remote non sono configurate.
4. Abilitare la Firebase Cloud Messaging API.
5. In **Impostazioni progetto > Account di servizio**, generare una chiave JSON e salvarla temporaneamente come `firebase-service-account.json` fuori dal repository oppure nella root (il nome è escluso da Git).
6. Codificare il JSON in Base64, così PowerShell non altera virgolette e caratteri di escape, salvarlo nei secrets Supabase e distribuire la funzione:

```
$firebase = [Convert]::ToBase64String(  
    [IO.File]::ReadAllBytes("$PWD\firebase-service-account.json")  
)  
npx supabase secrets set "FIREBASE_SERVICE_ACCOUNT_BASE64=$firebase"  
npx supabase functions deploy geofence-alert
```

Non inserire mai il service account nell'app, in `local.properties` o in Git. Dopo aver aggiunto `google-services.json`, ricostruire e reinstallare il ricevitore, aprirlo una volta e concedere il permesso notifiche. L'identificativo di installazione FCM viene registrato automaticamente in `receiver_push_tokens`.

Il trasmettitore invoca `geofence-alert` se l'avviso area è attivo o se una consegna è pendente. La RPC atomica `evaluate_geofence`, alla prima uscita, spegne l'avviso, attiva tracking e storico rapidi persistenti e accoda una sola notifica. Se FCM non conferma alcun invio, l'evento resta nel database e viene ritentato con backoff fino alla consegna.